A calculator for pure first-order resemblance: Cantor normal forms, finite patterns, and Lean proofs

23 October 2026
Working manuscript

Abstract

Carlson's pure first-order resemblance relation on the ordinals has an explicit arithmetic description below ε_0. We present a standalone calculator for that description, together with finite-pattern operations and a Lean development. The central operation computes the largest β such that $\alpha \leq_1 \beta$, using hereditary Cantor normal forms with compressed natural coefficients. We explain why the recursion terminates, why repeated summands must be combined by ordinal rather than commutative arithmetic, and how its result determines finite traces of the global relation. We also give a sharp quadratic bound on expanded output-tree size. The Lean development checks arithmetic normal-form and denotation results and finite-pattern laws; the identification of the arithmetic operation with recursive Σ_1-elementarity uses Carlson's published theorem. The Python implementation is separately tested against expanded reference calculations. Finite pattern enumeration supplies syntax and map tests, not an order-two ordinal assignment or a computation of a transfinite order type.

1 What the calculator computes

Two different orders occur in this subject. The usual ordinal order compares the positions of ordinals. A resemblance relation asks whether an earlier initial segment has enough of the finite configurations of a later one. Carlson defines the pure structure

$$\mathcal{R}_1 = (\mathrm{Ord}; \leq, \leq_1)$$

by recursion: $\alpha \leq_1 \beta$ means that $\alpha \leq \beta$ and the restriction to α is a Σ_1-elementary substructure of the restriction to β in this same relational language [1]. The reference to $\leq_1$ inside the language is part of the recursive definition. It is not ordinary elementarity in the language of linear order alone.

For a fixed α, its $\leq_1$-successors form an interval. Write $\mathrm{lh}(\alpha)$ for its largest element when that element exists. Below ε_0 there is a finite arithmetic recipe for this endpoint. Here ε_0 is the least fixed point of $\xi \mapsto \omega^\xi$, and a "finite recipe" means a terminating calculation on a finite ordinal notation, not that the ordinal denoted is finite. The result decides the relation by

$$\alpha \leq_1 \beta \quad \Longleftrightarrow \quad \alpha \leq \beta \leq \mathrm{lh}(\alpha). \tag{1}$$

The semantic justification is Carlson's arithmetic characterization, Theorem 2.5 of [1]. Wilken presents the endpoint recursion in a convenient Cantor-normal-form formulation in Proposition 1.1 of [2].

The distinction matters even in small examples. The calculator gives $\mathrm{lh}(\omega) = \omega + 1$, so $\omega \leq_1 \omega + 1$, whereas $\omega \not\leq_1 \omega + 2$. It does not merely test whether the two notations are in increasing order.

This paper explains an implementation of those published results. The recursion and its semantic identification are credited to their sources. The output-size proof, representation choices, finite map interface, and explicit division between tested and formalized claims are given here so that the package can be used without another project.

2 Hereditary Cantor normal forms

Every nonzero ordinal below ε_0 has a unique expression

$$a = \omega^{e_1} c_1 + \cdots + \omega^{e_k} c_k, \qquad e_1 > \cdots > e_k, \qquad 0 < c_i < \omega,$$

where each exponent is itself represented in the same way. Zero is the empty expression. The Python representation is a tuple of pairs (e_i, c_i), recursively using tuples for the exponents. Natural coefficients are ordinary integers and are stored without expanding c_i copies of the monomial.

A valid normal form has positive coefficients, recursively normal exponents, and strictly decreasing exponents at the outer level. These conditions are checked on programmatic inputs. For example, two separate consecutive ω^2 terms are not a canonical tuple; their canonical form combines their coefficients. The readable input layer constructs normal forms using the same arithmetic operations.

2.1 Addition and finite right multiplication

Ordinal addition is essential here. If the leading exponent of b is e, adding b to a removes the trailing monomials of a with exponent less than e, combines a remaining equal-exponent monomial if one exists, and then appends the remainder of b. Thus

$$1 + \omega = \omega, \qquad \omega + 1 > \omega.$$

The recursive normal-form comparison is lexicographic: compare leading exponents, then their coefficients, then the tails. On valid hereditary Cantor normal forms this is the usual ordinal order.

For a positive natural c, the operation $a \cdot c$ means repeated addition on the right. If $a = \omega^e d + t > 0$ is in normal form, then

$$a \cdot c = \omega^e (dc) + t.$$

Only the final copy retains the lower tail. In particular,

$$(\omega + 1) \cdot 2 = \omega \cdot 2 + 1. \tag{2}$$

Treating the coefficients as a reason to commute or distribute all summands would change the endpoint calculation.

3 The compressed endpoint recursion

Let $\mathrm{last}(a)$ be the final exponent of a nonzero normal form, and put $\mathrm{last}(0) = 0$. Define an auxiliary function P by

$$P(0) = 0, \tag{3}$$

$$P(\omega^e c + t) = (\omega^e + P(e)) \cdot c + P(t). \tag{4}$$

Define the executable endpoint function by

$$H(0) = 0, \qquad H(a) = a + P(\mathrm{last}(a)) \quad (a > 0). \tag{5}$$

All calls in (4) enter an exponent subtree or a proper tail of the input. The recursion therefore terminates structurally. Neither its termination argument nor its definition consults a resemblance oracle.

3.1 Why this is Wilken's formula

An additive-principal ordinal is a power of ω. Expand an exponent e as its weakly decreasing list of additive-principal summands $\rho_1, \ldots, \rho_m$, including repetitions from coefficients. For $\rho_i = \omega^{d_i}$, its computed endpoint is $\rho_i + P(d_i)$. Consequently (4) gives

$$P(e) = H(\rho_1) + \cdots + H(\rho_m). \tag{6}$$

The order of this sum is retained. Compression simply replaces a block of identical successive summands by finite right multiplication. Substituting (6) into (5) gives exactly the recursion in [2, Proposition 1.1]. Carlson's theorem then identifies $H(a)$ with $\mathrm{lh}(a)$, giving (1).

There are two correctness steps here. The first proves that a compressed algorithm denotes the displayed expanded ordinal sum. The second identifies that arithmetic sum with the recursively defined semantic relation. A successful program execution establishes neither theorem by itself. The formalization coverage is stated in Section 7.

3.2 Worked examples

Input a	Endpoint $H(a)$
0	0
7	7
ω	$\omega + 1$
ω^2	$\omega^2 + 2$
ω^ω	$\omega^\omega + \omega + 1$
$\omega^{\omega+1}$	$\omega^{\omega+1} + \omega + 2$
$\omega^{\omega\cdot 2}$	$\omega^{\omega\cdot 2} + \omega \cdot 2 + 1$

For the last example, the last exponent is $\omega \cdot 2$. Its two principal summands are both ω, whose endpoints are $\omega + 1$. Their sum is (2), which explains the final $+1$, rather than $+2$. This example tests the coefficient compression at a place where ordinary commutative arithmetic would give the wrong answer.

For a successor ordinal a, the final exponent is zero, so $H(a) = a$. For nonzero e, the recursion shows inductively that $P(e)$ has a positive finite final summand. Hence the endpoint of a nonzero limit below ε_0 is a successor. In particular,

$$H(H(a)) = H(a).$$

This is a useful consistency check on the arithmetic routine, although it is not a replacement for its denotation proof.

4 Finite patterns and maps

The second part of the package operates on finite relational syntax. Fix the underlying linear order $0 < \cdots < n-1$. A pure pattern of order $k \geq 1$ is represented by strict relations $R_1, \ldots, R_k$ on this set. Reflexive pairs are implicit. Starting with R_0 equal to the strict linear order, require at each level:

1. $R_i \subseteq R_{i-1}$;
2. R_i is transitive;
3. if $xR_{i-1}yR_{i-1}z$ and xR_iz, then xR_iy.

These are the finite nesting and respect conditions used in the pure pattern setting; see the introduction of [2]. The implementation checks them on the finite relations. Its order-two objects remain finite syntax: constructing one does not assign its least ordinal realization.

4.1 Coverings preserve; induced embeddings also reflect

A covering from A to B is an increasing injection f that preserves every higher relation:

$$xR_i^A y \implies f(x)R_i^B f(y).$$

An induced embedding additionally reflects the relations, replacing the implication by an equivalence. On the fixed finite linear order, all increasing injections can be enumerated by their increasing image lists. Thus a search through the $\binom{|B|}{|A|}$ possible image sets is a complete finite map search, with no candidate when $|A| > |B|$.

For example, let A be a two-point pattern with no strict R_1 edge, and let B have the edge $0R_11$. The identity on the underlying two-point order is a covering $A \to B$: there are no positive R_1 facts in A to violate. It is not an induced embedding, because it does not reflect the new edge of B. Conversely, there is no covering $B \to A$.

Both notions have identities and compose. Increasing injections compose, and relation preservation passes through the intermediate pattern. For induced embeddings the two reflection implications compose as well. These elementary laws are useful checks on an implementation because confusing preservation with reflection changes the research problem.

4.2 Markers and finite traces

A pattern may carry a distinguished point. It need not be the maximum. A map is required to preserve that point only when marker preservation is explicitly requested; that request requires both markers to exist. The number of higher relations determines the pattern's order, regardless of where its marker lies.

For strictly increasing normal-form coordinates $a_0 < \cdots < a_{n-1}$, the calculator can form the finite trace whose strict first relation is

$$iR_1j \quad \iff \quad i < j \ \text{ and } \ a_j \leq H(a_i).$$

This is the restriction of the global pure relation, using Carlson's semantic theorem. It is not obtained by recomputing elementarity inside the finite list itself. For instance the trace on $\{\omega, \omega+1\}$ retains a genuine edge of $\mathcal{R}_1$. Recomputing a recursive elementary-substructure relation on a finite linear order gives a different object, since a new element beyond a finite last parameter is witnessed by an existential formula.

5 What the finite enumeration counts

The implementation enumerates the strict relations satisfying the stated laws on the fixed underlying order. Up to five points it gives:

Number of points	0	1	2	3	4	5
Order one	1	1	2	5	14	42
Order two	1	1	3	14	84	596

The empty pattern is included. Fixing the linear order eliminates nontrivial automorphisms, so these are finite syntactic isomorphism-type counts. They are not counts of covering-equivalence classes, minimal ordinal realizations, or ordinal ranks. In particular, their numerical growth cannot verify a conjecture about a transfinite order type.

The search is exhaustive only for its supplied finite size. Its combinatorial cost grows quickly; the command-line interface therefore keeps large enumerations behind an explicit size limit. No empirical failure to find a pattern or a large covering is used as an unbounded negative theorem.

6 A sharp expanded-tree size bound

Compact storage size and the mathematical norm of a notation are different quantities. Define the expanded hereditary monomial-tree weight by

$$\mathrm{W}(0) = 0, \qquad \mathrm{W}(\omega^e c + t) = c(1 + \mathrm{W}(e)) + \mathrm{W}(t). \tag{7}$$

Repeated exponent subtrees are charged repeatedly, and coefficients are unary for this measure. Put $\mathrm{T}(n) = n(n+1)/2$.

Theorem 6.1. *For every normal form a below ε_0,*

$$\mathrm{W}(P(a)) \le \mathrm{T}(\mathrm{W}(a)), \qquad \mathrm{W}(H(a)) \le \mathrm{T}(\mathrm{W}(a)).$$

The second bound is attained for every natural input weight.

Proof. Ordinal addition deletes a final segment and possibly combines two equal-exponent coefficient blocks. It introduces no new exponent subtree. Therefore

$$\mathrm{W}(u+v) \le \mathrm{W}(u) + \mathrm{W}(v), \qquad \mathrm{W}(u \cdot c) \le c\,\mathrm{W}(u).$$

Also $\mathrm{T}(x) + \mathrm{T}(y) \le \mathrm{T}(x+y)$ and $c\,\mathrm{T}(x) \le \mathrm{T}(cx)$ for natural c, x, y.

For the first inequality, use structural induction. Zero is immediate. Write $a = \omega^e c + t$, $r = \mathrm{W}(e)$, and $s = \mathrm{W}(t)$. The recursion and induction hypotheses give

$$\begin{aligned} \mathrm{W}(P(a)) &\le c(1 + r + \mathrm{W}(P(e))) + \mathrm{W}(P(t)) \\ &\le c(1 + r + \mathrm{T}(r)) + \mathrm{T}(s) \\ &= c\,\mathrm{T}(r+1) + \mathrm{T}(s) \\ &\le \mathrm{T}(c(r+1) + s) = \mathrm{T}(\mathrm{W}(a)). \end{aligned}$$

For nonzero a put $n = \mathrm{W}(a)$ and $e = \mathrm{last}(a)$. At least one monomial of a contains that exponent, so $1 + \mathrm{W}(e) \le n$. Hence

$$\mathrm{W}(H(a)) \le n + \mathrm{W}(P(e)) \le n + \mathrm{T}(n-1) = \mathrm{T}(n).$$

At zero the same assertion is equality.

For sharpness define $t_0 = 0$ and $t_{n+1} = \omega^{t_n}$. Then $\mathrm{W}(t_n) = n$, and direct induction in the recursion gives

$$H(t_n) = t_n + t_{n-1} + \cdots + t_1 \quad (n \ge 1).$$

The displayed summands are in decreasing Cantor normal form. Their expanded weights sum to $n + (n-1) + \cdots + 1 = \mathrm{T}(n)$. Use zero for $n = 0$. □

This theorem concerns output size under (7). It is not a quadratic running-time claim for arbitrary Python integers, a bound in binary coefficient length, or a theorem comparing ordinal notations with minimum pattern size. Those require separate cost models and mathematical arguments.

7 Lean coverage and the semantic boundary

The Lean package pins its toolchain and Mathlib revision [5]. Its arithmetic development uses Mathlib's `ONote` syntax and normal subtype `NONote`. Positive coefficients in that syntax are typed as positive natural numbers. Normal forms carry proof obligations rather than relying on unchecked tuple conventions.

The endpoint development separates three tasks: preserving normality, proving that compressed arithmetic has its intended ordinal denotation, and identifying that denotation with resemblance. Its normal-form and denotation results establish the first two tasks. In particular, the principal-summand specification expands repetitions only in the mathematical statement, and the endpoint-recursion theorem equates the computed ordinal representation with the sum in (6).

The finite-pattern development checks the relation conditions and the corresponding map laws in a finite model. It keeps a covering distinct from an induced embedding and makes marker preservation explicit. Concrete finite examples exercise those definitions. The accompanying coverage guide lists the actual public Lean declarations and their checked axiom dependencies.

The Lean finite existence checker enumerates all functions between the two finite vertex types and filters them by the map predicate. Python uses increasing image lists instead. These are complete searches for the same specified maps, but no equality of running times or formal cross-language refinement is asserted.

Carlson's recursive Σ_1-elementarity characterization is cited as a published mathematical theorem. This package does not introduce it as a new Lean axiom or claim a formalization of its entire proof. Similarly, the proof of Theorem 6.1 is given in this paper; its presence is not a claim that every argument here has been translated into Lean. Formalization coverage is attached to named declarations, not inferred from the existence of a `.lean` file.

The Python and Lean implementations also remain different programs. Kernel checking the Lean definitions does not verify the Python interpreter or compiler. The Python test suite uses independent expanded recursions, noncommutative arithmetic examples, malformed-input tests, and finite map cases. Cross-implementation examples add evidence that the two implementations agree on their common representation.

8 Using and reproducing the package

The project contains the Python package, standard-library tests, a command-line interface, the Lean source and Lake configuration, this manuscript, and a local PDF build script. Its readable ordinal input is a small specified grammar; it is parsed as data rather than evaluated as Python code. Machine-readable output includes a versioned format. The precise command syntax and examples are in `USAGE.md`.

The basic Python check is

```
python3 -m unittest discover -s tests -v
```

For a fresh Lean installation, use the toolchain and dependency revisions specified in the `lean/` directory. After acquiring the pinned dependencies and their compiled cache, run the project build. The optional `scripts/check_lean.py` compiles named modules serially with explicit time and memory limits; it does not update dependencies. Its locally borrowed cache is a development convenience, not a repository dependency.

The test and build records identify what was actually run. A successful bounded test checks those instances. General claims above rest on the specified mathematical proofs or the named Lean theorems, according to their stated coverage. Resource refusal does not certify a negative mathematical answer.

9 Research role and directions for reuse

The calculator was developed as a first-order base case for work on finite patterns and ordinal notations. A motivating question in Carlson's earlier cohabitating-trees manuscript concerns the computability and order types of patterns ordered by their least ordinal coverings [4, Section 5]. That author draft conjectures that the order-two type κ_2 is the proof-theoretic ordinal of $\mathrm{KP}\ell_0$. The finite order-two syntax interface here is preparation for such questions, not their ordinal evaluator. Related marked pure-order-two notation results are already supplied by [3]; the formulation of the source question must be checked before treating it as open.

Three immediate uses are concrete. First, endpoint and finite-map calculations provide regression cases for more elaborate pattern algorithms. Second, the expanded weight and Theorem 6.1 support controlled-size calculations, where changing the coefficient encoding can change the statement. Third, finite traces supply explicit examples separating inherited resemblance from a relation recomputed inside a finite structure. A catalogue of finite certificate tasks can combine those operations without claiming to measure an agent's intrinsic proof-theoretic ordinal.

Extensions need to respect the language. Pure $\mathcal{R}_1$, additive patterns, and the first relation inside simultaneous higher-order structures are different semantic objects. Wilken's introduction [2] explains where the simple pure repetition ceases to describe the first relation of $\mathcal{R}_2$. Reusing the Cantor-normal- form arithmetic is often possible; reusing the resemblance decision without a transfer theorem is not. A complete order-two ordinal assignment, a higher-pattern comparison procedure, and a Bachmann–Howard termination-order implementation each require additional mathematics.

References

[1] T. J. Carlson, *Ordinal arithmetic and* Σ_1*-elementarity*, Archive for Mathematical Logic **38** (1999), 449–460.

[2] G. Wilken, *Pure* Σ_2*-elementarity beyond the core*, Annals of Pure and Applied Logic **172** (2021), article 103001. `https://arxiv.org/abs/1710.01870v5`.

[3] G. Wilken, *Pure patterns of order 2*, Annals of Pure and Applied Logic **169** (2018), 54–82. `https://arxiv.org/abs/1608.08421`.

[4] T. J. Carlson, *Generalizing Kruskal's theorem to pairs of cohabitating trees*, author manuscript dated 6 October 2008, Section 5. Published version: Archive for Mathematical Logic **55** (2016), 37–48, `https://doi.org/10.1007/s00153-015-0457-4`. The concluding conjecture passage cited here is in the earlier author manuscript and is not reproduced in the final journal body.

[5] The mathlib Community, *Mathlib4*, source revision `360da6fa`, especially `Mathlib.SetTheory.Ordinal.Notation`.